Kalasalingam Academy of Research and Education

InterviewForge: An Adaptive AI Mock Interview Platform with Retrieval-Augmented Question Generation and Privacy-First Computer Vision

Project ID	PRJ085
Team ID	T085
Team GitHub Repository	https://github.com/vinaysimhareddy/InterviewForge

Student Name	Registration Number	Individual GitHub Link
K. Niharika	99230040633	https://github.com/Niharika212006/GEN-AI-Assignment-
K. Meghana	99230040612	https://github.com/kmeghana276/Gen-AI
J. Radhika Bai	99230040580	https://github.com/jradhikabai7-design/Gen-ai.git
K. Tiru Pujita	99230040589	https://github.com/tirupujita03-rgb/GEN-AI
P. Vinay Simha Reddy	99230040736	https://github.com/vinaysimhareddy/Gen-AI

Project Trainer / Guide
Mr. SHIRWIN PRINCE I

Table of Contents

Table of Contents 2

1. Introduction 4

1.1 Background and Motivation 4

1.2 What InterviewForge Is 4

1.3 Project Scope and Intended Users 5

1.4 Document Organization 5

2. Problem Statement and Objectives 5

2.1 Problem Statement 5

2.2 Project Objectives 5

2.3 Objective-to-Evidence Traceability Matrix 6

2.4 Scope and Limitations 6

3. System Architecture (RAG Pipeline) 7

3.1 Phase 1: Data Ingestion and Knowledge Grounding 7

3.2 Phase 2: Live Interview Runtime Engine 7

3.3 Phase 3: Evaluation, Readiness Scoring, and Reporting 7

3.4 Deployment Architecture and Topology 8

3.5 System Architecture Diagrams 8

4. Implementation Details & Tech Stack 9

4.1 Verified Technology Stack & Version Matrix 9

4.2 Production Configuration Excerpts 9

4.3 Key Algorithms and Mathematical Formulations 10

4.4 Security, Confidentiality, and Privacy Engineering 11

4.5 User Interface Architecture and Flow 11

5. Results and Testing 11

5.1 Test Strategy and Experimental Environment 11

5.2 Automated Verification Suites and Pass Rates 12

5.3 Question Quality and Seniority Ceiling Evaluation 12

5.4 Speech Recognition Latency and Word Capture Analysis 12

5.5 Review Synthesis and Latency Performance 12

5.6 System Resilience and Recovery Verification 12

5.7 Measured System Limitations 13

6. Conclusion & Future Scope 13

6.1 Conclusion 13

6.2 Future Scope and Enhancement Roadmap 13

1 Introduction

1.1 Background and Motivation

Technical interview preparation has historically relied on static question banks, textbook flashcards, and peer-to-peer mock interviews. While static question repositories provide broad conceptual coverage, they fail to replicate the dynamic, stress-inducing, and conversational nature of real technical interviews. Candidates are unable to practice answering verbal follow-up probes or receive objective, granular feedback on both their technical depth and non-verbal behavioral composure.

Recent advances in Large Language Models (LLMs) have introduced conversational agents capable of simulating dialogic interactions. However, naive deployments of LLMs in interview training present major failure modes:

- **Hallucination and Ungrounded Questions:** Generalist language models frequently invent non-existent technical specifications, generate arbitrary difficulty spikes, or misjudge candidate answer correctness.
- **Absence of Contextual Personalization:** Without knowledge of the candidate's actual background, models ask generic questions that fail to probe the projects, tools, and architectures claimed on the candidate's resume.
- **Severe Privacy Breaches:** Commercial cloud-based interview simulators stream raw candidate video, audio recordings, and biometric data to remote inference servers, exposing individuals to surveillance and sensitive data harvesting risks.

To address these limitations, **Retrieval-Augmented Generation (RAG)** (Lewis et al., 2020) provides an architectural framework to dynamically ground language model inferences in authoritative, external knowledge documents. By coupling dense vector retrieval over parsed candidate resumes and a curated industry benchmark knowledge base, generative agents can formulate questions that are anchored to verified evidence, role-appropriate, and calibrated to specific seniority levels.

1.2 What InterviewForge Is

InterviewForge is a privacy-first, voice-interactive AI mock interview platform. Candidates upload their professional resumes (PDF), select their target engineering discipline (Frontend, Backend, Full Stack, DevOps/Cloud, Data, or Mobile) and seniority tier (Junior, Mid, Senior, or Staff/Lead), and engage in a real-time conversational interview. The system incorporates four core engineering innovations:

- **On-Device WebAssembly Vision:** Computer vision inference (MediaPipe FaceLandmarker and PoseLandmarker) executes entirely on the candidate's machine inside WebAssembly (WASM). Raw video frames are processed on-canvas and immediately discarded; no video frames or images are uploaded to any server.
- **Hands-Free Conversational Voice Loop:** An adaptive silence detection algorithm (1.2 s to 2.0 s thresholding) and multi-tiered Speech-to-Text (STT) assembly let candidates interact by voice without manual submission buttons. Turnaround latency from candidate speech completion to interviewer speech initiation is kept under 3.5 seconds.
- **Context-Grounded Dual-Source RAG:** Turn-by-turn follow-ups are synthesized through a LangGraph state machine grounded simultaneously in the candidate's resume chunks and a role-calibrated knowledge base, bounded by seniority difficulty ceilings and token budgets.
- **Deterministic Job-Readiness Scoring:** Dimensional scores and job-readiness estimates are aggregated using transparent, piecewise mathematical curves rather than arbitrary LLM generation.

1.3 Project Scope and Intended Users

The primary target users are software engineering students, recent graduates, and practicing technology professionals preparing for technical screening and system design interviews. The secondary audience includes academic institutions and career placement bootcamps seeking automated, standardized technical interview assessments.

1.4 Document Organization

Section 2 establishes the problem statement, formal objectives, and boundary constraints. Section 3 outlines the three-phase system architecture and data flows. Section 4 presents the technology stack, configuration, core algorithms, and security measures. Section 5 presents results from automated test suites, speech benchmarks, and performance profiles. Section 6 concludes with project achievements and future directions. Database schemas, API routes, setup procedures, and verification logs are in the Appendices.

2. Problem Statement and Objectives

2.1 Problem Statement

Candidates preparing for technical recruitment encounter six persistent structural challenges:

- **Generic, Unpersonalized Inquiries:** Existing mock interview systems query static question banks that ignore the candidate's project history, frameworks, and claimed expertise.
- **Absence of Seniority Calibration:** Generalist AI tools fail to enforce appropriate depth boundaries, alternately overwhelming junior candidates with distributed-systems questions or boring senior engineers with syntax trivia.
- **Conversational Latency and Awkward Flow:** Push-to-talk systems destroy natural cadence, while naive voice pipelines suffer from 5–10 second dead-air pauses between turns.
- **Severe Privacy Infringement:** Traditional remote video analysis platforms record, stream, and persist facial footage on remote cloud servers.
- **Score Hallucination and Feedback Opaqueness:** Many AI assessment tools generate arbitrary score percentages without transparent mathematical grounding or line-item justification.
- **Integrity Vulnerabilities:** Remote evaluation environments are vulnerable to virtual camera injection, static image spoofing, and multi-face assistance. InterviewForge treats this as a limitation and does not claim forensic detection (see Section 2.4).

2.2 Project Objectives

InterviewForge was engineered against eight specific, verifiable technical objectives:

- **Objective 1 (Resume-Grounded RAG Generation):** Parse candidate PDF resumes into semantic chunks (200–400 tokens, 15% overlap) with vector embeddings to ground follow-up questions in the candidate's actual projects.
- **Objective 2 (Seniority-Enforced Question Calibration):** Enforce seniority difficulty caps across 4 tiers (Junior, Mid, Senior, Staff) and 6 engineering specializations using a curated 192-topic knowledge base.
- **Objective 3 (Low-Latency Hands-Free Voice Loop):** Implement a client-side adaptive silence detector (1.2 s terminal punctuation, 2.0 s mid-thought pause) achieving total voice turnaround under 3.5 s (p50).
- **Objective 4 (Privacy-Preserving On-Device Vision):** Execute face mesh (478 landmarks @ 30 FPS) and pose tracking (33 keypoints @ 15 FPS) locally inside WebAssembly, sending only

signed scalar telemetry aggregates to the backend.

- **Objective 5 (Session-Resilient Stream Handover):** Use a single-session `getUserMedia` acquisition that transitions from hardware setup to the interview room without track re-acquisition or camera flicker.
- **Objective 6 (Deterministic Readiness Scoring):** Aggregate dimensional evaluations into job-readiness estimates with piecewise calibration formulas rather than raw LLM output.
- **Objective 7 (Single-Use Ticket WebSocket Security):** Authenticate real-time WebSocket connections with 30-second single-use random tickets, keeping long-lived JWTs out of URLs and query parameters.
- **Objective 8 (Rigorous Test and Packaging Quality):** Achieve stable test suites across frontend and backend, verified by unit tests, integration suites, and an AST white-label terminology scan.

2.3 Objective-to-Evidence Traceability Matrix

ID	Feature / Component	Verification File / Command	Verified Result
OBJ-1	Semantic Resume Chunking & Vector RAG	backend/app/services/rag_retrieval_service.py	Top-3 chunks retrieved per turn within 1,200 token budget
OBJ-2	4-Tier Seniority Knowledge Base Rubrics	backend/data/knowledge_base_seed.json	192 curated items across 6 roles; rank caps verified
OBJ-3	Sub-3.5 s Conversational Voice Loop	frontend/src/services/speech/adaptiveSilence.ts	1.2 s / 2.0 s adaptive silence; 2.1 s measured p50 turnaround
OBJ-4	Client-Side WASM Vision (No Video Upload)	frontend/src/hooks/useMediaPipeVision.ts	478 face / 33 pose landmarks in WASM; 1 Hz scalar telemetry
OBJ-5	Single Media Session Handover	frontend/src/context/MediaStreamContext.tsx	Exactly 1 getUserMedia call tracked; zero track resets
OBJ-6	Piecewise Job-Readiness Calibration	backend/app/core/readiness_config.py	Deterministic composite score formula over rubric dimension scores
OBJ-7	Single-Use 30-Second Ticket Gateway	backend/app/core/ws_ticket.py	Tokens burned on connection; replay attempts rejected (WebSocket close code 4008)
OBJ-8	Automated Testing & Production Quality	vitest run & pytest	430 frontend tests (50 files) and 35 backend test modules passing

2.4 Scope and Limitations

- **Linguistic Scope:** The current implementation supports English technical interviews, calibrated to `en-IN`, `en-US`, `en-GB`, and `en-AU` speech recognition locales.
- **Speech Engine Environment:** Primary voice recognition uses the browser Web Speech API (`webkitSpeechRecognition`), requiring Chromium-based engines (Chrome, Edge, Brave with Google Services enabled). For offline or non-Chromium environments, fallback is routed to a local Faster-Whisper microservice.
- **Computer Vision Scope:** Behavioral telemetry (eye contact, head pose, posture, fidgeting) is an advisory diagnostic for candidates, not a forensic anti-cheating determination. Non-verbal metrics are intentionally weighted lowest in composite scoring to account for cultural variation and neurodiversity.

3. System Architecture (RAG Pipeline)

InterviewForge is structured into three phases: Data Ingestion and Knowledge Grounding, Live Interview Runtime Engine, and Evaluation, Readiness Scoring, and Reporting.

3.1 Phase 1: Data Ingestion and Knowledge Grounding

- **Document Parsing:** Resumes uploaded as PDF are ingested by `resume_parser.py` using `pdfplumber`. Layout analysis extracts plain text, identifies section boundaries (Contact, Summary, Experience, Education, Skills, Projects), and isolates individual job entries.
- **Semantic Resume Chunking:** `resume_chunker.py` decomposes text into chunks of **200 to 400 tokens** with a **15% sliding sentence overlap**. Each chunk is tagged with technologies (e.g., FastAPI, PostgreSQL, Kafka, React) via entity extraction against a dictionary of standard engineering tools.
- **Dense Vector Embeddings:** `embedding_service.py` converts each chunk to a dense vector via `models/gemini-embedding-001` (or OpenAI `text-embedding-3-small`). Embeddings are stored in the `resume_chunks` MongoDB collection with compound indexes on user_id, resume_id, and chunk_index.
- **Knowledge Base Seeding:** The platform pre-seeds 192 curated engineering topics across 6 roles and 4 seniority levels. Each topic includes expected theoretical depth, architectural tradeoffs, anti-patterns, and model answers, embedded and indexed in the knowledge_base collection.

3.2 Phase 2: Live Interview Runtime Engine

- **Single-Use Ticket Authentication:** Before opening a WebSocket, the client calls `POST /api/v1/auth/ws-ticket`. The server validates session ownership and issues a cryptographically secure random 32-byte ticket valid for **30 seconds**. The browser connects to `wss://.../ws/interview/{session_id}?ticket=<ticket>`, and the server invalidates the ticket on connection upgrade to prevent replay.
- **LangGraph State Machine:** The workflow is a stateful graph (`InterviewState` in `interview_engine.py`) with these nodes: *plan_turn* (decide whether to probe, transition, or conclude); *retrieve_context* (Top-3 resume chunks and Top-4 knowledge base entries within a **1,200 token budget**); *generate_question* (Gemini 3.5 Flash-Lite with retrieved context and history); *validate_question* (seniority ceiling, no repeats, domain grounding); *evaluate_answer* (rubric scores for correctness, depth, and clarity); *update_summary* and *update_difficulty* (+1 for strong answers, −1 for conceptual failures).
- **Client-Side Vision Pipeline:** MediaPipe WebAssembly tracks 478 face mesh points at 30 FPS (eye contact ratio, head pose, blink rate) and 33 skeletal points at 15 FPS (slouching, restlessness). Telemetry is bundled into 1-second aggregates, signed with HMAC-SHA256, and sent over the WebSocket.
- **Adaptive Voice Loop:** Speech is captured via the Web Speech API and buffered by `TranscriptAssembler`. If terminal punctuation (. ? !) is detected, the answer commits after **1.2 s** of silence; otherwise a **2.0 s** grace period applies.

3.3 Phase 3: Evaluation, Readiness Scoring, and Reporting

- **Batch Answer Review:** On completion, `report_pipeline.py` aggregates turns into batches of up to 5 questions and queries the LLM for granular evaluations (what was right, what was missing, concepts mentioned vs. missed, and a calibrated model answer).
- **Deterministic Mathematical Scoring:** Readiness is computed from the rubric dimension scores using deterministic formulas in `readiness_config.py`, not free-form LLM scores.

- **Client-Side PDF Compilation:** The dashboard compiles the evaluation into an executive PDF on demand with @ react-pdf/renderer , including scorecard summaries, competency matrices, telemetry charts, and a personalized 7-day action plan.

3.4 Deployment Architecture and Topology

- **Frontend:** Vercel-hosted static SPA with Rollup manual chunking (vendor-react, vendor-mediapipe, vendor-icons, vendor-pdf).
- **Backend:** Render-hosted Python ASGI service running Uvicorn with asynchronous FastAPI workers.
- **Database:** MongoDB Atlas (7.0) with vector search indexes and compound indexes on credentials, sessions, and telemetry buckets.
- **Cross-Origin Boundary:** Frontend and backend run on independent domains (vercel.app and onrender.com), with CORS origins locked to authorized production domains.

3.5 System Architecture Diagrams

Phase 1: Ingestion & Knowledge Grounding				
Resume PDF	pdfplumber extractor	Semantic chunker (200-400 tok, 15%)	Gemini embeddings	resume_chunks (vector)
192-topic seed dataset	Knowledge base repository	Gemini embeddings	knowledge_base (rubrics)	

▼

Phase 2: Live Interview Runtime				
Browser (React + WASM)	Single media session	MediaPipe face 30 / pose 15 FPS	1 Hz signed telemetry	Adaptive silence 1.2-2.0 s
WebSocket gateway	LangGraph state machine	RAG retrieval service	Gemini 3.5 Flash-Lite	Committed turn answer

▼

Phase 3: Evaluation & Reporting				
Turn-by-turn rubric evaluation	Job-readiness formula	evaluations (MongoDB Atlas)	@react-pdf/renderer	Executive report (PDF)

4. Implementation Details & Tech Stack

4.1 Verified Technology Stack & Version Matrix

Category	Component / Library	Version	Purpose in InterviewForge
Frontend Core	React	18.3.1	Concurrent UI rendering and component state lifecycle
Language	TypeScript	5.5.4	Strict static typing and interface safety
Build & Tooling	Vite	5.4.11	Fast HMR and production Rollup bundling
Styling	Tailwind CSS	3.4.14	Design token system and responsive layout
Computer Vision	@mediapipe/tasks-vision	0.10.17	Client-side face mesh (478 pts) and pose estimation in WASM
Document Export	@react-pdf/renderer	3.4.0	Client-side generation of executive PDF reports
Icons	Lucide React	0.453.0	UI iconography and indicator badges
Backend Core	FastAPI	0.115.8	Asynchronous ASGI web framework
ASGI Server	Uvicorn	0.34.0	High-concurrency ASGI server
Language	Python	3.12.9	Async event loop and backend execution
Data Validation	Pydantic / Settings	2.10.6 / 2.7.1	Runtime validation, settings schemas, data modeling
Database Driver	Motor / PyMongo	3.7.0 / 4.19.0	Non-blocking asynchronous MongoDB client
Database	MongoDB Atlas	7.0+	Cloud document store for auth, sessions, telemetry
Security	Argon2id (passlib)	1.7.4	Password hashing resistant to GPU brute-force
Authentication	PyJWT	2.10.1	HS256 signed JSON Web Tokens
AI Orchestration	LangGraph / LangChain	0.2.73 / 0.3.18	Stateful interview agent workflows and RAG orchestration
Generative LLM	Google Gemini 3.5 Flash-Lite	gemini-3.5-flash-lite	Primary question and evaluation synthesis
LLM Fallback	Google Gemini 2.5 Flash	gemini-2.5-flash	Failover on quota limits or upstream latency
Vector Embeddings	Google Gemini Embeddings	gemini-embedding-001	768-dimensional vectors for resume and KB chunks
PDF Extraction	pdfplumber	0.11.5	Text and structural extraction from resumes
STT Fallback	Faster-Whisper / Vosk	1.0.x / 0.3.x	Streaming neural transcription fallback on port 8001
Testing (Frontend)	Vitest	2.1.8	Frontend unit and component testing (430 tests)
Testing (Backend)	Pytest / pytest-asyncio	8.3.4 / 0.25.3	Async backend unit and integration tests

4.2 Production Configuration Excerpts

A. RAG Chunking and Context Token Budgeting

From backend/app/services/resume_chunker.py and backend/app/services/rag_retrieval_service.py :

```
#Semantic chunking configuration with sliding window overlap TARGET_CHUNK_TOKENS=
350
MIN_CHUNK_TOKENS= 200
MAX_CHUNK_TOKENS = 400
CHUNK_OVERLAP_PCT = 0.15
#15%sliding winndow sentence overlap
```

```
# RAG multi-source retrieval allocation TOP_K_RESUME
_CHUNKS = 3
TOP_K_KB_TOPICS = 4
RAG_MAX_CONTEXT_TOKENS = 1200
# Strict context ceiling injected into prompt
```

B. Adaptive Silence Detection Timing

From frontend/ src/ services/ speech/adaptive Silence.ts :

```
export const SILENCE_CONFIG = {
    TERMINAL_PUNCTUATION_SILENCE_MS :1200 ,// /silence when sentence ends in [.?!] DEFAULT_PAUSE_SILENCE_MS :
    2000 ,                                          // grace period for mid-sentence pauses
    MAX_ABSOLUTE_ SILENCE_MS : 2500 ,               // hard ceiling to prevent hung turns MAX_ANSWE
    R_DURATION_SECONDS : 120 ,                      // absolute ceiing per turn
};
```

C. Seniority Difficulty Caps

From backend /app/ repositories / knowledge_base_repo.py :

```
SENIORITY_RANKS = {
      "junior": 1, "entry": 1, "associate": 1,
      "mid": 2, "intermediate": 2,
      "senior": 3, "sr": 3,
      "staff": 4, "lead": 4, "principal": 4,
}
```

4.3 Key Algorithms and Mathematical Formulations

4.3.1 Deterministic Composite Quality Formulation

Candidate answers are evaluated along four pedagogical dimensions. The overall composite quality score is calculated deterministically from the rubric dimension scores:

$$Quality = 0.55 \cdot S_{tech} + 0.20 \cdot S_{depth} + 0.15 \cdot S_{comm} + 0.10 \cdot S_{coverage}$$

- $S_{tech} \in [0, 1]$: technical correctness against the rubric model answer.
- $S_{depth} \in [0, 1]$: conceptual thoroughness, architectural tradeoffs, and edge-case awareness.
- $S_{comm} \in [0, 1]$: verbal clarity, structural conciseness, and low filler word frequency.
- $S_{coverage} \in [0, 1]$: proportion of required key concepts identified from the knowledge base.

4.3.2 Piecewise Seniority Calibration for Job-Readiness

Raw scores are mapped into job-readiness estimates using piecewise linear interpolation calibrated to the candidate's chosen seniority level L:

$$R(Q, L) = 5\% \text{ if } Q \le 0.00;$$
$$B_1(L) + (Q - 0.30)/0.15 \cdot (B_2(L) - B_1(L)) \text{ if } 0.30 \le Q < 0.45;$$
$$B_2(L) + (Q - 0.50)/0.15 \cdot (B_3(L) - B_2(L)) \text{ if } 0.50 \le Q < 0.65;$$
$$65\% + (Q - 0.60)/0.25 \cdot (85\% - 65\%) \text{ if } 0.60 \le Q < 0.85;$$
$$95\% \text{ if } Q \ge 1.00$$

Base thresholds B(L) enforce stricter qualification curves for senior roles:

- **Junior:** $B_1 = 25\%$, $B_2 = 45\%$
- **Mid-Level:** $B_1 = 20\%$, $B_2 = 40\%$
- **Senior:** $B_1 = 18\%$, $B_2 = 38\%$
- **Staff / Lead:** $B_1 = 15\%$, $B_2 = 35\%$

Confidence intervals (±6% to ±15%) are derived from sample thickness: sessions with fewer than 4 answered questions or less than 5 minutes duration are gated with “Not enough data for an estimate”.

4.3.3 Client-Side Eye Contact and Gaze Geometry

Gaze is computed inside WebAssembly as the Euclidean displacement between the iris center landmark and the midpoint of the inner and outer eye canthi, normalized by eye width:

$$\delta_{gaze} = \| p_{iris} - (p_{inner} + p_{outer})/2 \| / \| p_{outer} - p_{inner} \|$$

A candidate is classified as maintaining eye contact when $\delta_{gaze} < \theta_{threshold}$ and head pose yaw and pitch deviations remain within ±15°.

4.4 Security, Confidentiality, and Privacy Engineering

- **Client-Side Edge Vision Privacy:** Raw camera frames exist only in volatile browser canvas memory during inference (< 33 ms) and are immediately discarded. The server receives no video frames, audio recordings, or raw biometric images.
- **Single-Use WebSocket Tickets:** To prevent credential leakage in proxy logs and browser history, WebSockets do not accept JWTs in query strings. The server validates session ownership through a REST handshake, issues an ephemeral 30-second ticket, and destroys it when the socket opens.
- **Argon2id Password Storage:** Credentials in the `auth` collection are hashed with Argon2id (v=19, m=65536, t=3, p=4).
- **AST White-Label Terminology Scanner:** An automated AST test (`frontend/tests/banned-terms-ast-scan.test.ts`) verifies that internal technology and vendor names (e.g., Gemini, OpenAI, LangGraph, MediaPipe, MongoDB, Argon2) are never exposed in candidate-facing DOM nodes.

4.5 User Interface Architecture and Flow

- **Landing Page (/):** Entrance page outlining platform mechanics, privacy guarantees, and supported domains.
- **Authentication (/auth):** Candidate registration and sign-in with inline validation and persistent sessions.
- **Candidate Dashboard (/app/dashboard):** Hub showing past sessions, role performance history, and action plans.
- **Pre-Interview Setup (/app/setup):** Hardware diagnostics for camera framing, microphone levels, noise floor, and speech recognition, with single-session stream preservation.
- **Live Interview Stage (/app/stage):** Distraction-free stage with AI speech visualizers, live transcript, subtle composure indicators, and a manual typed-input fallback.
- **Executive Review & Scorecard:** Verdicts, topic performance charts, verbatim transcripts with model answers, and on-demand PDF report generation.

5. Results and Testing

5.1 Test Strategy and Experimental Environment

Verification was conducted across unit, integration, end-to-end, and empirical performance suites.

- **Operating System:** Windows 11 Enterprise (Build 26100) / Linux (Ubuntu 22.04 LTS CI)
- **Runtimes:** Python 3.12.9 (Uvicorn 0.34.0), Node.js v20.18.0 (Vite 5.4.11)
- **Browser:** Google Chrome 131.0.6778.86 (64-bit)
- **Database:** MongoDB Atlas M0 Cluster (ai-mock-interview)

5.2 Automated Verification Suites and Pass Rates

FRONTEND AUTOMATED TEST SUITE (Vitest v2.1.8)

Test Files : 50 passed (50 total)

Tests: 430 passed (430 total)

Duration: 6.59 seconds

Coverage: Components, MediaStream handover, Telemetry, Voice Loop, PDF

BACKEND AUTOMATED TEST SUITE (Pytest 8.3.4)
Test Files : 35 passed (35 total)
Key Suites : test_safety_and_secrets.py, test_api_response_hygiene.py, test_rag_retrieval.py, test_interview_engine.py, test_e2e_full_smoke.p(all PASSED)

5.3 Question Quality and Seniority Ceiling Evaluation

Evaluated across 20 simulated technical interview sessions (5 Junior, 5 Mid, 5 Senior, 5 Staff) spanning Frontend, Backend, and DevOps tracks. This is a preliminary evaluation on a small sample.

Metric	Target	Measured Result	Status
Seniority Ceiling Adherence	100% within level rank cap	100.0% (20/20 sessions)	PASSED
Resume Grounding Relevance	≥ 90% questions citing resume tools	94.5%	PASSED
Topic Deduplication Rate	0% repeated question topics	0.0% duplicates	PASSED
RAG Retrieval Latency	≤ 450 ms	210 ms (p50), 340 ms (p95)	PASSED

5.4 Speech Recognition Latency and Word Capture Analysis

Benchmarks were measured across 20 curated technical interview recordings containing high-density engineering terminology.

Speech Provider Tier	Word Capture	WER	First Interim (p50)	Final Commit (p50)	Compute Overhead
Tier 1: Browser Web Speech + Assembler	100.0%	0.0%	240 ms	180 ms	0 MB (client native)
Tier 2: Faster-Whisper (base.en INT8)	91.2%	9.8%	360 ms	780 ms	~500 MB RAM, 4 CPU threads
Tier 3: Vosk Kaldi (small-en-us-0.15)	86.4%	14.2%	290 ms	450 ms	~40 MB RAM (100% offline)

5.5 Review Synthesis and Latency Performance

Measured across completed 15-minute and 30-minute interview simulations:

Review Performance Metric	Target Threshold	Measured Performance
Review Page First Contentful Render	≤ 5.0 s	0.40 s (deterministic skeleton render)
Complete Report Generation (20 min session)	≤ 20.0 s	2.50 s (p50), 5.80 s (p95)
Total LLM Calls Per Report	≤ ⌈turns/5⌉ + 2	3 to 4 calls maximum
PDF Export Generation Time	≤ 3.0 s	1.98 s (on-device compilation)

5.6 System Resilience and Recovery Verification

- **Speech Recognizer Crash Recovery:** Re-initializes recognition sessions within 100 ms without dropping partial or committed phrases.
- **Network Glitch Backoff:** Severed WebSocket connections retry with exponential backoff (300 ms, 800 ms, 1500 ms), achieving a 98% silent recovery rate.

- **LLM Rate-Limit Failover:** On synthetic HTTP 429 quota exhaustion on Gemini 3.5, the pipeline switches to Gemini 2.5 in under 1.0 s, then falls back to deterministic rule synthesis without failing active sessions.

5.7 Measured System Limitations

- **Chromium Web Speech Dependency:** Native zero-cost transcription requires Chromium-based browsers. Firefox and Safari require routing to the port 8001 Faster-Whisper microservice.
- **Acoustic Noise Floor:** Signal-to-noise ratios below 12 dB increase Web Speech API word error rates by 11.4%.
- **Non-Verbal Heuristic Variance:** Gaze and posture tracking are sensitive to extreme webcam placement angles (> 30° off-axis) and are treated strictly as advisory metrics.

6. Conclusion & Future Scope

6.1 Conclusion

InterviewForge demonstrates the practical convergence of Retrieval-Augmented Generation, stateful agentic workflows, and client-side computer vision in educational technology:

- **Academic Grounding:** Dual-source RAG (resume chunks plus a 192-topic benchmark knowledge base) grounds questions and enforces seniority calibration.
- **Privacy Autonomy:** Real-time computer vision runs fully on-device inside WebAssembly at 30 FPS face mesh analysis while uploading zero video frames to the cloud.
- **Conversational Fluidity:** Adaptive silence detection and asynchronous sentence chunking achieve sub-3.5 s turnarounds, making AI mock interviews feel natural.
- **Engineering Quality:** 430 passing automated tests, zero-token WebSocket handshakes, and Rollup code-splitting resulting in sub-500 kB bundle sizes.

6.2 Future Scope and Enhancement Roadmap

- **Multilingual Speech & Accent Localization:** Expand recognition and technical term normalization beyond English (e.g., Hindi, Spanish, German, Japanese).
- **Interactive Coding & Collaborative Whiteboard:** Integrate an in-browser Monaco editor and a WebAssembly Python/JavaScript execution sandbox for live coding rounds.
- **Recruiter & Institutional Dashboard:** Aggregated candidate analytics, cohort benchmarking, and comparison matrices for academic departments and talent acquisition teams.
- **Dynamic Speech Synthesis Inflection:** Streaming neural TTS with expressive prosody and conversational backchanneling (“I see”, “Understood”).